seL4 Core and seL4 Core Platform (sel4cp)

Page Content

Status

- Overview:
  - https://www.youtube.com/watch?v=khcjH77riGA
- Repos:
  - SDK: https://github.com/BreakawayConsulting/sel4cp
  - docs: https://github.com/BreakawayConsulting/platform-sel4-docs
  - seL4 kernel fork with the "sel4core" working branch: https://github.com/BreakawayConsulting/seL4/tree/sel4core
- Status
  - currently it's a proposal driven by Gernot and Ben Leslie (Breakaway Consulting, https://brkawy.com)
  - there is a commercial project: LAOT
    - https://trustworthy.systems/projects/TS/laot
    - https://eds.power.on.net/laot-pub/index
  - the RFC process is ongoing, it is supposed to eventually make this an seL4 Foundation project
    - https://sel4.atlassian.net/browse/RFC-5 (The seL4 Core Platform)
    - https://sel4.atlassian.net/browse/RFC-6 (The seL4 Core)
- open projects
  - port seL4 Core Platform to QEMU to allows exploring it
  - https://www.unsw.edu.au/engineering/student-life/undergraduate-research-opportunities/advertised-taste-research-areas/automatic-verification-sel4-core-platform

Technical Overview

Acronyms

- CC: communication channel
- MR: message register
- PD: protection domain
- PP: protected procedure
- PPC : protected procedure call

seL4 Core

- distribution package of seL4 kernel and everything needed to build on top of it
- minimum set of dependencies to build any project using the kernel
- contents:
  - kernel binary
  - C library wrapping syscalls + associated header files
  - lib containing a platform-agnostic subset of the C library
    - aim: smaller than ffreestanding even
    - sel4runtime GIT repo already is a step towards this
- CAmkES is to be constructed on top of that (rather than depending on the seL4 CMake project)
- having a seL4 binary releases nicely aligns with with our MiG-V "seL4 in ROM story"

seL4 Core Platform

- runtime on top of seL4
- aims to be a much leaner interface for building "seL4 systems"
- uses a precompiled kernel binary (seL4 Core), so a verified kernel can be used
  - **ToDo:** adapt our SDK to support this use case also, we need this anyway to use it with seL4 is running from ROM on MiG-V
- uses XML files to define components and the system architecture
  - **ToDo:** is this really needed, can't we use the CAmkES language here?
    - What would be needed in the CAmkES language to support this
    - Does is make sense to write a converter? There are python script processing the XML, can we hook into them?
- Protection Domain
  - basically a lightweight process
  - C-Space, V-Space, Scheduling Context (MCS-Kernel), Priority
  - single threaded (maybe multi-threadding support in the future)

        - static, ie known at build time, no dynamic loading
        - functions
            - init procedure (invoke once on startup)
            - notification procedure (invoked by signal)
            - protected procedure
                - client-server model
                - other PD can invoke this as a "library call"
                - caller trusts callee
                - called must run at higher priority, guarantees there are no deadlocks
                - nesting is possible A B C
- Communication channel
    - always between exactly 2 PDs
    - shared memory object that can be referenced
    - MR holds arguments
    - unidirectional or bidirectional
    - read and write
    - seL4 hard limit: 64 CC per PD (limited by badge ID)
    - signal other PD (signalling multiple times may result on one signal only)
    - invoke protected procedure in other PD
    - order "should" be driven by sender PD priority (may not be the case in first implementation)
- Virtual machine as abstraction of PD
    - extra execution mode: guest mode